

Course: Image Processing 31651
Micro Project 2 (MP2) : Usage of Filtration
Date of submission : 15/09/2024

	ID (4 last digits)	Shorten Name	Photo of the student	
Student #1	1950	shienfeld		
Student #2	2210	pony		
Student #3	7939	akimov		

Micro Project 2 (MP2) : Usage of Filtration – the given task: (part 1)

2

Create Test Image and blur it by using Convolution

- (1 point) By using MS Paint
create Gray BMP VGA Image (640x480) Tim1.bmp containing 12 lines of the text:
"Hello from **ABCD-EFGH-IJKL**" (Replace RED)
* Font size of every line is differ (from small (say, 8 or less) to large)
- (5 points) By using **reworked 2D** 5x5 and 7x7 filters **BLUR** Image Tim1.bmp
by using convolution with **Fast Gaussian 1D** Filters

1/273

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

1/1003

0	0	1	2	1	0	0
0	3	13	22	13	3	0
1	13	59	97	59	13	1
2	22	97	159	97	22	2
1	13	59	97	59	13	1
0	3	13	22	13	3	0
0	0	1	2	1	0	0

Resulted images names: Tim1LPF5c.bmp
and Tim1LPF7c.bmp

Select font sizes so that after the blurring
at least 2 lines will be hardly readable and 3 lines unreadable

Micro Project 2 (MP2) : Usage of Filtration – the given task: (part 2)

3

Blur test Image by using FFT and “restore it”

3 (2 points) Use filtration with FFT to blur Tim1.bmp by using Gaussian Filter in FD. Select size of the Gaussian Filter so that **at least 6 lines will be hardly readable and unreadable.**

Resulted image: Tim1LPFF.bmp

4 (6 points) In attempt to “restore” above blurred images (by using convolution filters), “invent” at least two “High Frequency Enhancing Filters” and apply them to above blurred images. Add to “restored images” prefix HFEF1c and HFEF2c

5 (4 points) In attempt to “restore” above blurred images by using FFT, “invent” High Frequency Enhancing Filter” in FD and apply it to above blurred images. Add to “restored images” prefix HFEFf

6 (6 points) Properly number all images and profiles in the report and explain your results by referring all the images: what was good and wat was not good.

7 (6 points) Recreate grayImage13bmp from Assignment 13 (TV Test Image) with sizes 640x480. Repeat above steps with this reworked image. To retain naming scheme, rename it to **Tim2.bmp**

Pay attention : that presentation is ordered in the following order : first introductory part where we present and explain the functions we used in the project – stand alone way (every function independently from the main flow) – and then the formal results for 1-7 sub tasks. They will be marked MP2.#

That function applies a convolution operation between a given image (src) and a filter kernel (filter) to produce a filtered image (dest)

```
125 //DoFiltationByConvolution is a given function by samuel - page 5 in filtration by convolution lecture
126 void DoFiltationByConvolution(unsigned char src[][NUMBER_OF_COLUMNS], unsigned char dest[][NUMBER_OF_COLUMNS],
127                               double* filter, int filter_size)
128 {
129     // Calculate half-width and half-height dynamically based on the filter size
130     int filter_half_size = filter_size / 2;
131
132     // Perform convolution
133     for (int row = filter_half_size; row < NUMBER_OF_ROWS - filter_half_size; row++)
134     {
135         for (int column = filter_half_size; column < NUMBER_OF_COLUMNS - filter_half_size; column++)
136         {
137             double summa = 0;
138             for (int y = -filter_half_size; y <= filter_half_size; y++)
139             {
140                 for (int x = -filter_half_size; x <= filter_half_size; x++)
141                 {
142                     // Perform convolution with the unsharp masking kernel
143                     summa += filter[(y + filter_half_size) * filter_size + (x + filter_half_size)] * src[row + y][column + x];
144                 }
145             }
146
147             // Clip values to the 0-255 range
148             summa = std::min(255.0, std::max(0.0, summa));
149
150             // Write the result into the destination image
151             dest[row][column] = static_cast<unsigned char>(summa);
152         }
153     }
154 }
```

Due to the nature of the function (nested loops) we prefer to explain it step by step in a big space- hence the explanation will be in the next slide!

MP2.0 – introduction part 1.2 - the function “DoFiltationByConvolution”

5

```
129 // Calculate half-width and half-height dynamically based on the filter size
130 int filter_half_size = filter_size / 2;
```

First, calculates the filter half size through the filter size – we do it for determining the "radius" of the filter



Then we use 4 nested loops – the two first loops are Iterating Over Each Pixel in the Source Image (Excluding Borders)

Then declare a variable to accumulates the weighted sum of neighboring pixel values based on the filter coefficients.

Then, in the two last for loops, applying the Convolution Filter – the results are being added

```
132 // Perform convolution
133 for (int row = filter_half_size; row < NUMBER_OF_ROWS - filter_half_size; row++)
134 {
135     for (int column = filter_half_size; column < NUMBER_OF_COLUMNS - filter_half_size; column++)
136     {
137         double summa = 0;
138         for (int y = -filter_half_size; y <= filter_half_size; y++)
139         {
140             for (int x = -filter_half_size; x <= filter_half_size; x++)
141             {
142                 // Perform convolution with the unsharp masking kernel
143                 summa += filter[(y + filter_half_size) * filter_size + (x + filter_half_size)] * src[row + y][column + x];
144             }
145         }
146     }
147 }
```

For each pixel at position (row, column), the function calculates the convolution sum (summa) by multiplying each element of the filter kernel with the corresponding pixel in the src image. The sum is accumulated for the entire kernel.

MP2.0 – introduction part 1.3 - the function “DoFiltationByConvolution”

6

next is clipping the Result to the Valid Pixel Range (0-255).

The purpose: Ensures that the resulting pixel value stays within the valid grayscale range.

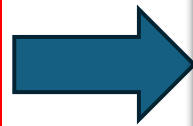
If summa is less than 0.0, it is set to 0.0. If summa is greater than 255.0, it is set to 255.0. Otherwise, summa remains unchanged.

```
147 // Clip values to the 0-255 range
148 summa = std::min(255.0, std::max(0.0, summa));
149
150 // Write the result into the destination image
151 dest[row][column] = static_cast<unsigned char>(summa);
```

Finally : assign the Computed Value to the Destination Image

Short remainder of 2D convolution:

$$I'(x, y) = \sum_{i=-m}^m \sum_{j=-n}^n K(i, j) * I(x + i, y + j)$$



$I(x, y)$ is the intensity of the pixel at position (x, y) in the src image.
 $K(i, j)$ is the value of the filter kernel at position (i, j) .
 $I'(x, y)$ is the intensity of the pixel at position (x, y) in the dest image.
 m and n are the half-width and half-height of the kernel, respectively

The 2D convolution is a dimensional expansion to the well known 1D convolution as we studies in digital signal processing Couse

MP2.0 – introduction part 2 - the function “Convert”

7

The Convert function scales and normalizes the floating-point image data (**floatImage**) into the 0-255 range, which is suitable for 8-bit grayscale images.

```
157 //convert is a given function by samuel - page 59 in FFT part 1 lecture
158 void Convert(tFloat floatImage[][NUMBER_OF_COLUMNS], unsigned char byteOriginal[][NUMBER_OF_COLUMNS], tFloat minVal, tFloat maxVal)
159 {
160     tFloat* ptrToFloatImage = floatImage[0];
161     tFloat c, b, temp;
162     if (minVal == maxVal)
163     {
164         c = 1; b = 0;
165     }
166     else
167     {
168         c = 255.0 / (maxVal - minVal);
169         b = -c * minVal;
170     }
171
172     unsigned char* ptrToByteImage = byteOriginal[0];
173     ptrToFloatImage = floatImage[0];
174
175     for (int pixel = 0; pixel < NUMBER_OF_COLUMNS * NUMBER_OF_ROWS; pixel++)
176     {
177         temp = c * (*ptrToFloatImage++) + b;
178         if (temp < 0) temp = 0;
179         if (temp > 255) temp = 255;
180         *ptrToByteImage++ = (unsigned char)((int)temp);
181     }
182 }
```

The function calculates two factors, c and b, which are used to linearly transform the values in floatImage to the desired 0-255 range.

The function then iterates over every pixel in the **floatImage**, applying the linear transformation defined by c and b.

For each pixel, the formula used is $\text{temp} = c * (*\text{ptrToFloatImage}++) + b$. This calculates the new pixel value in the normalized range.

The value is then clamped to ensure it stays within the 0-255 range. If temp is less than 0, it is set to 0; if greater than 255, it is set to 255.

MP2.0 – introduction part 3 - the function “OptimalConvert”

8

That function is designed to take an image represented in the frequency domain (stored in a floating-point array **floatImage**) and convert it into a grayscale image represented as an 8-bit unsigned char array (**byteOriginal**).

```
185 //OptimalConvert is a given function by samuel - page 60 in FFT part 1 lecture
186 void OptimalConvert(tFloat floatImage[][NUMBER_OF_COLUMNS], unsigned char byteOriginal[][NUMBER_OF_COLUMNS])
187 {
188     tFloat* ptrToFloatImage = floatImage[0];
189     tFloat MinVal, MaxVal, temp;
190     // Find MinMax
191     MinVal = *ptrToFloatImage;
192     MaxVal = MinVal;
193
194     for (int pixel = 0; pixel < NUMBER_OF_COLUMNS * NUMBER_OF_ROWS; pixel++)
195     {
196         temp = *ptrToFloatImage++;
197         if (MinVal > temp) MinVal = temp;
198         if (MaxVal < temp) MaxVal = temp;
199     }
200     Convert(floatImage, byteOriginal, MinVal, MaxVal);
201 }
```

The function scans through the **floatImage** to find the minimum (**MinVal**) and maximum (**MaxVal**) values. This step is crucial because the pixel values after FFT and subsequent operations might span a wide range and normalizing them is necessary for visualization.

The function then calls **Convert**, passing the **MinVal** and **MaxVal** it found. The **Convert** function normalizes the values in **floatImage** so that they fit within the 0-255 range, which is suitable for a grayscale image.

This conversion is crucial because, after various operations in the frequency domain, the pixel values might not be in a range that can be directly displayed or interpreted.

MP2.0 – introduction part 4 - the function “DoFFT”

9

The DoFFT function converts an image between the spatial domain and the frequency domain. In the frequency domain, the image data is represented by two components: the real part (imageRe) and the imaginary part (imageIm).

```
204 //DoFFT is a given function by samuel - page 61 in FFT part 1 lecture
205 void DoFFT(tFloat imageRe[][NUMBER_OF_COLUMNS], tFloat imageIm[][NUMBER_OF_COLUMNS], int exponentSign,int normalizationScalingType)
206 {
207
208     tFloat* ptrToRe = imageRe[0];
209     tFloat* ptrToIm = imageIm[0];
210
211     tInteger arrayOfDimensions[2];
212     arrayOfDimensions[0] = NUMBER_OF_COLUMNS;
213     arrayOfDimensions[1] = NUMBER_OF_ROWS;
214
215     PrimeFFTN(2,
216               arrayOfDimensions,
217               ptrToRe,
218               ptrToIm,
219               exponentSign,
220               normalizationScalingType);
221 }
222
```

The core of the DoFFT function is the call to PrimeFFTN, which is a specialized FFT function provided in the **PrimeFFTN.h** library (was given to use in the website of the course). This function performs the actual FFT calculation based on the provided parameters:

2 indicates that the FFT is being performed on a 2D array.

arrayOfDimensions specifies the dimensions of the array.

ptrToRe and **ptrToIm** point to the real and imaginary parts of the image data.

exponentSign determines the direction of the FFT.

normalizationScalingType controls how the result is scaled.

MP2.0 – introduction part 5 - the function “ShiftHalfSize”

10

That function shifts the quadrants of the image so that the zero frequency component (also known as the DC component) is centered.

```
224 //ShiftHalfSize is a given function by samuel - page 62 in FFT part 1 lecture
225 void ShiftHalfSize(tFloat floatImage[][NUMBER_OF_COLUMNS])
226 {
227     tFloat* floatImage_ptr= floatImage[0];
228     for (int row = 0; row < NUMBER_OF_ROWS; row++)
229     {
230         for (int column = 0; column < NUMBER_OF_COLUMNS; column += 2)
231         {
232             floatImage[row][column] *= -1;
233         }
234     }
235
236     for (int row = 0; row < NUMBER_OF_ROWS; row += 2)
237     {
238         for (int column = 0; column < NUMBER_OF_COLUMNS; column++)
239         {
240             floatImage[row][column] *= -1;
241         }
242     }
243 }
```

It rearranges the output of the FFT such that the low frequencies are centered, which is often more intuitive for analysis and visualization.

The **ShiftHalfSize** function is conceptually similar to the `fftshift` function in MATLAB.

The primary purpose of DoFiltrationInFD is to apply a filter (stored in floatFilter) to an image that has already been transformed into the frequency domain.

```
246 //DoFiltrationInFD is a given function by samuel - page 63 in FFT part 1 lecture
247 void DoFiltrationInFD(tFloat floatRe[][NUMBER_OF_COLUMNS], tFloat floatIm[][NUMBER_OF_COLUMNS], tFloat floatFilter[][NUMBER_OF_COLUMNS])
248 {
249     tFloat* ptrToRe = floatRe[0];
250     tFloat* ptrToIm = floatIm[0];
251     tFloat* ptrToFilter = floatFilter[0];
252
253     for (int pixel = 0; pixel < NUMBER_OF_ROWS * NUMBER_OF_COLUMNS; pixel++)
254     {
255         *ptrToRe++ *= *ptrToFilter;
256         *ptrToIm++ *= *ptrToFilter++;
257     }
258 }
259
```

The filtering is done by element-wise multiplication of the frequency domain representation of the image (stored in **floatRe** and **floatIm** arrays) with the filter.

After applying the filter in the frequency domain, the next step is to perform an inverse FFT (using **DoFFT** with a negative exponent) to transform the filtered image back into the spatial domain. The result is a filtered version of the original image.

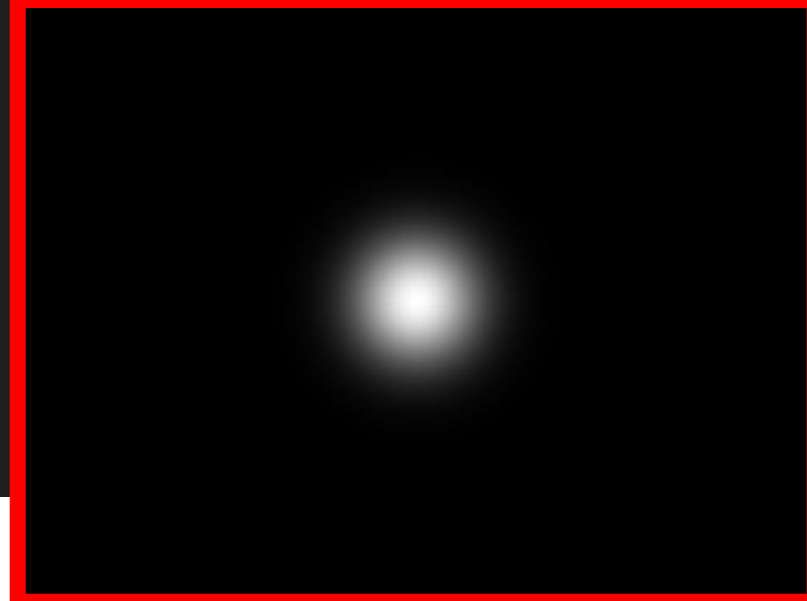
MP2.0 – introduction part 7 - the function “CreateGreyGaussian”

12

The purpose of the CreateGreyGaussian function is to create a synthetic grayscale image that simulates the effect of a Gaussian function applied over a 2D plane.

```
262 void CreateGreyGaussian(unsigned char(*image)[NUMBER_OF_COLUMNS], double SigmaX, double SigmaY) {  
263     int midy = NUMBER_OF_ROWS / 2;  
264     int midx = NUMBER_OF_COLUMNS / 2;  
265  
266     for (int row = 0; row < NUMBER_OF_ROWS; row++) {  
267         for (int column = 0; column < NUMBER_OF_COLUMNS; column++) {  
268             double xa = column - midx;  
269             xa /= SigmaX;  
270             double ya = row - midy;  
271             ya /= SigmaY;  
272  
273             double answer = 255.0 * (exp(-xa * xa - ya * ya));  
274  
275             image[row][column] = (unsigned char)(unsigned)(answer);  
276         }  
277     }  
278 }
```

Example of the outcome – from the work() function – with the parameters we chose for blurring



In this case the parameters are: filter_size / 10.0, filter_size / 10.0

The purpose of the function is to be the basis of the blurring process using FFT – LPF gaussian blurring

$$\text{Gaussian Formula: Intensity} = 255 \times e^{-\left(\frac{(x-\text{midx})^2}{2\sigma_x^2} + \frac{(y-\text{midy})^2}{2\sigma_y^2}\right)}$$

```
281 void DoGaussianFiltration(unsigned char* src, double* filter, int filter_size) //filteration using convolution - x direction and then y direction
282 {
283     int half_size = (filter_size - 1) / 2;
284     double summa;
285
286     memset(temp_dest, 0, sizeof(temp_dest));
287
288     // x direction
289     for (int row = 0; row < NUMBER_OF_ROWS; row++) {
290         for (int column = half_size; column < NUMBER_OF_COLUMNS - half_size; column++) {
291             summa = 0;
292             for (int x = -half_size; x <= half_size; x++) {
293                 summa += filter[half_size + x] * (double)*(src + row * NUMBER_OF_COLUMNS + (column + x));
294             }
295             if (summa < 0) summa = 0;
296             if (summa > 255) summa = 255;
297             *(temp_dest[0] + row * NUMBER_OF_COLUMNS + column) = (unsigned char)summa;
298         }
299     }
300
301     // Clear Source !!!
302     for (int i = 0; i < NUMBER_OF_ROWS * NUMBER_OF_COLUMNS; i++) {
303         src[i] = 0;
304     }
305
306     // y direction
307     for (int row = half_size; row < NUMBER_OF_ROWS - half_size; row++) {
308         for (int column = 0; column < NUMBER_OF_COLUMNS; column++) {
309             summa = 0;
310             for (int y = -half_size; y <= half_size; y++) {
311                 summa += filter[half_size + y] * *(temp_dest[0] + (row + y) * NUMBER_OF_COLUMNS + column);
312             }
313             if (summa < 0) summa = 0;
314             if (summa > 255) summa = 255;
315             *(src + row * NUMBER_OF_COLUMNS + column) = (unsigned char)summa;
316         }
317     }
```

The purpose of this function is perform a linear convolution in TD between a gaussian filter (a kernel in certain filter size) and a source input image



Next slide we will present the main idea of the function – of how it convolve the filter with the src in an efficient way because of the gaussian filter

The function performs Gaussian filtering on a grayscale image using a separable convolution approach.

This method involves two consecutive one-dimensional (1D) convolutions—first along the x-axis (horizontal direction) and then along the y-axis (vertical direction).

This approach is computationally more efficient than applying a two-dimensional (2D) convolution directly: This reduces computational complexity from $O(n^2)$ to $O(2n)$, where n is the filter size.

Calculate the half-size of the filter to handle boundary conditions.

Clear the temporary buffer (temp_dest) to prepare for storing intermediate results.

Apply the 1D Gaussian filter across each row and Store the filtered results in temp_dest.

Apply the 1D Gaussian filter across each column using the intermediate results from temp_dest and then Store the final filtered values back into src.

MP2.0 – introduction part 9 - the function “CreateLowPassFilter”

15

The function computes a 2D **Butterworth** low-pass filter based on the provided cutoff frequency and order.

```
379 void CreateLowPassFilter(tFloat(*floatFilter)[NUMBER_OF_COLUMNS], int cutoff, int order)
380 {
381     int centerX = NUMBER_OF_COLUMNS / 2;
382     int centerY = NUMBER_OF_ROWS / 2;
383
384     // Loop to compute the low-pass filter
385     for (int row = 0; row < NUMBER_OF_ROWS; row++)
386     {
387         for (int col = 0; col < NUMBER_OF_COLUMNS; col++)
388         {
389             double distance = sqrt(pow(row - centerY, 2) + pow(col - centerX, 2));
390             floatFilter[row][col] = 1.0 / (1.0 + pow(distance / cutoff, 2 * order));
391         }
392     }
393 }
```

The Butterworth filter is known for its maximally flat magnitude response in the passband, meaning it doesn't have ripples and provides smooth attenuation. By modifying the order, you can control how quickly the filter transitions from passband to stopband, allowing flexibility based on application needs.

Butterworth Low-Pass Filter Formula:

$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0}\right)^{2n}}$$



1. $H(u, v)$ is the filter coefficient at position (u, v) .
2. $D(u, v)$ is the distance from the center.
3. D_0 is the cutoff frequency.
4. n is the order of the filter.

```
320 //work() is the a general function that includes all the steps that necessary for conducting FFT and filtering in FD in the blurring process
321 void Work(unsigned char ProccesIMG[][NUMBER_OF_COLUMNS], int filter_size)
322 {
323     cout << "FFT Filtration in nearly Plain C" << endl;
324
325     // Step 1: Input image processing - FFT in preparation for the filtration process in DoFiltrationInFD
326     for (int i = 0; i < NUMBER_OF_ROWS; i++)
327     {
328         for (int j = 0; j < NUMBER_OF_COLUMNS; j++)
329         {
330             floatRe[i][j] = ProccesIMG[i][j];
331             floatIm[i][j] = 0; // Ensure imaginary part is initialized to 0
332         }
333     }
334
335     // Step 2: Apply FFT preprocessing (centering)
336     ShiftHalfSize(floatRe);
337     ShiftHalfSize(floatIm);
338
339     // Step 3: Perform FFT to convert the image to the frequency domain
340     DoFFT(floatRe, floatIm, FORWARD_FFT, NORMALIZE_BY_SQRT);
341
342     // Step 4: Prepare the Gaussian filter
343     // Modify this line to make filter_size impact the Gaussian filter
344     CreateGreyGaussian(GrayImage1, filter_size / 10.0, filter_size / 10.0); // Make filter_size affect sigma
345     StoreGrayImageAsGrayBmpFile(GrayImage1, "GreyGaussianFilter.bmp");
346
347     // Convert the GrayImage1 to floatFilter (as it's likely in 8-bit unsigned char format) -- very important!!
348     for (int i = 0; i < NUMBER_OF_ROWS; i++)
349     {
350         for (int j = 0; j < NUMBER_OF_COLUMNS; j++)
351         {
352             floatFilter[i][j] = (tFloat)GrayImage1[i][j] / 255.0; // Normalize to 0-1 range
353         }
354     }
```

The function is a comprehensive image processing routine that applies a High-Frequency Enhancing Filter (HFEF) to a grayscale image using Frequency Domain Filtering.

The work() function uses most of the functions we explained in the previews parts of MP2.0

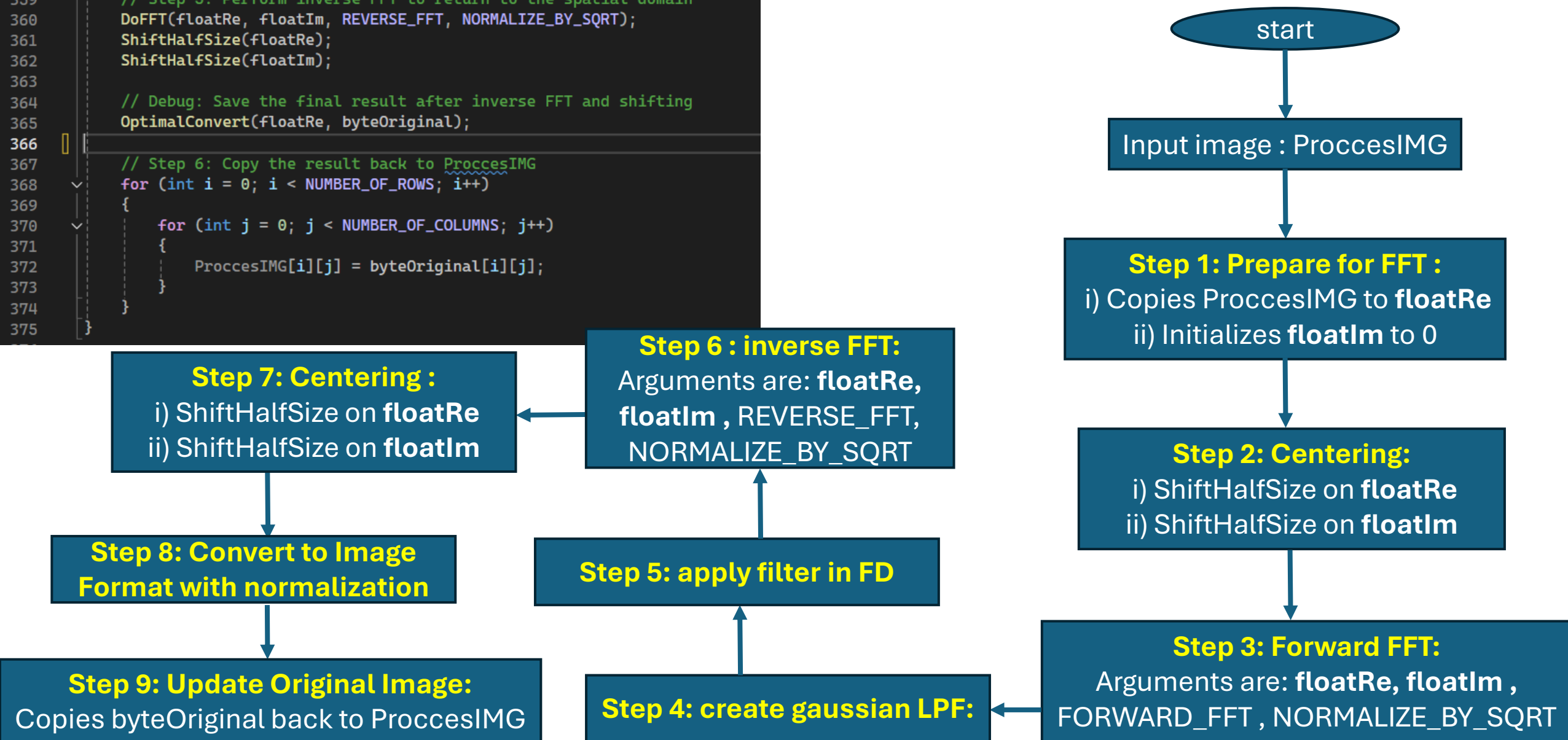
Next slide – an block diagram of the function and the rest of the code

MP2.0 – introduction part 10.2 - the function “Work”

17

```
356 // Apply the Gaussian filter in the frequency domain
357 DoFiltrationInFD(floatRe, floatIm, floatFilter);
358
359 // Step 5: Perform inverse FFT to return to the spatial domain
360 DoFFT(floatRe, floatIm, REVERSE_FFT, NORMALIZE_BY_SQRT);
361 ShiftHalfSize(floatRe);
362 ShiftHalfSize(floatIm);
363
364 // Debug: Save the final result after inverse FFT and shifting
365 OptimalConvert(floatRe, byteOriginal);
366
367 // Step 6: Copy the result back to ProccesIMG
368 for (int i = 0; i < NUMBER_OF_ROWS; i++)
369 {
370     for (int j = 0; j < NUMBER_OF_COLUMNS; j++)
371     {
372         ProccesIMG[i][j] = byteOriginal[i][j];
373     }
374 }
375 }
```

Block diagram of the work() function:



MP2.0 – introduction part 11.1 - the function

“CreateHighFrequencyEnhancingFilter”

The function is designed to generate a High-Frequency Enhancing Filter (HFEF) in the frequency domain

```

396 void CreateHighFrequencyEnhancingFilter(tFloat(*floatFilter)[NUMBER_OF_COLUMNS], int cutoff, int order, float boost_factor)
397 {
398     // First, create the LPF
399     CreateLowPassFilter(floatFilter, cutoff, order);
400
401     // Then, compute the HFEF
402     for (int row = 0; row < NUMBER_OF_ROWS; row++)
403     {
404         for (int col = 0; col < NUMBER_OF_COLUMNS; col++)
405         {
406             floatFilter[row][col] = (1.0f + boost_factor) - boost_factor * floatFilter[row][col];
407         }
408     }
409 }

```

The function begins by invoking CreateLowPassFilter, which generates a Butterworth Low-Pass Filter (LPF) and stores it in the **floatFilter** matrix.

Then, the function transforms the created LPF into High-Frequency Enhancing Filter (HFEF)

An HPF attenuates low frequencies and allows high frequencies to pass through. An HFEF amplifies high frequencies while keeping low frequencies unchanged or less amplified.

The high-frequency enhancing filter can be defined mathematically as:

$$H_{HFEF}(u, v) = (1 + a) - a * H_{LPF}(u, v)$$

That formula leverages the LPF we easily create to ensure the filter has the desired characteristics to fulfil its purposes.

MP2.0 – introduction part 11.2 - the function “CreateHighFrequencyEnhancingFilter”

The function we created use several parameters in accordance with the mathematical background of the filter

```
graph TD; A[The function we created use several parameters in accordance with the mathematical background of the filter] --> B[Cutoff frequency – determines the frequency at which the filter transitions from low to high frequencies]; A --> C[Filter order - controls the sharpness of the filter's transition between low and high frequencies. The lower the order – the smoother (gradual) transition]; A --> D[Boost factor – determines the amount by which high frequencies are amplified. The lower the boost factor , the subtle enhancement to high frequencies];
```

Cutoff frequency – determines the frequency at which the filter transitions from low to high frequencies

Filter order - controls the sharpness of the filter's transition between low and high frequencies. The lower the order – the smoother (gradual) transition

Boost factor – determines the amount by which high frequencies are amplified.
The lower the boost factor , the subtle enhancement to high frequencies

```

412 void WorkHFEF(unsigned char ProccesIMG[][NUMBER_OF_COLUMNS],int cutoff,int order, float boost_factor)
413 {
414     cout << "FFT Filtration with High Frequency Enhancing Filter (HFEF)" << endl;
415
416     // Step 1: Prepare the image for FFT
417     for (int i = 0; i < NUMBER_OF_ROWS; i++)
418     {
419         for (int j = 0; j < NUMBER_OF_COLUMNS; j++)
420         {
421             floatRe[i][j] = ProccesIMG[i][j];
422             floatIm[i][j] = 0; // Ensure imaginary part is initialized to 0
423         }
424     }
425
426     // Step 2: Apply centering to the image
427     ShiftHalfSize(floatRe);
428     ShiftHalfSize(floatIm);
429
430     // Step 3: Perform forward FFT on the input image for being able to filter it using HFEF in FD
431     DoFFT(floatRe, floatIm, FORWARD_FFT, NORMALIZE_BY_SQRT);
432
433     //Step 4: create a high enhancing frequencies filter
434
435     CreateHighFrequencyEnhancingFilter(floatFilter, cutoff, order, boost_factor);
436
437     // Visualize the HFEF
438     Convert(floatFilter, byteOriginal, 1.0f, 1.0f + boost_factor);
439     StoreGrayImageAsGrayBmpFile(byteOriginal, "HFEF.bmp");
440
441     // Step 5: Apply the HFEF in the frequency domain
442     DoFiltrationInFD(floatRe, floatIm, floatFilter);
443
444     // Step 6: Perform inverse FFT on the filtered image in FD
445     DoFFT(floatRe, floatIm, REVERSE_FFT, NORMALIZE_BY_SQRT);
446     ShiftHalfSize(floatRe);
447     ShiftHalfSize(floatIm);

```

The function is a comprehensive image processing routine that applies a High-Frequency Enhancing Filter (HFEF) to a grayscale image using Frequency Domain Filtering.

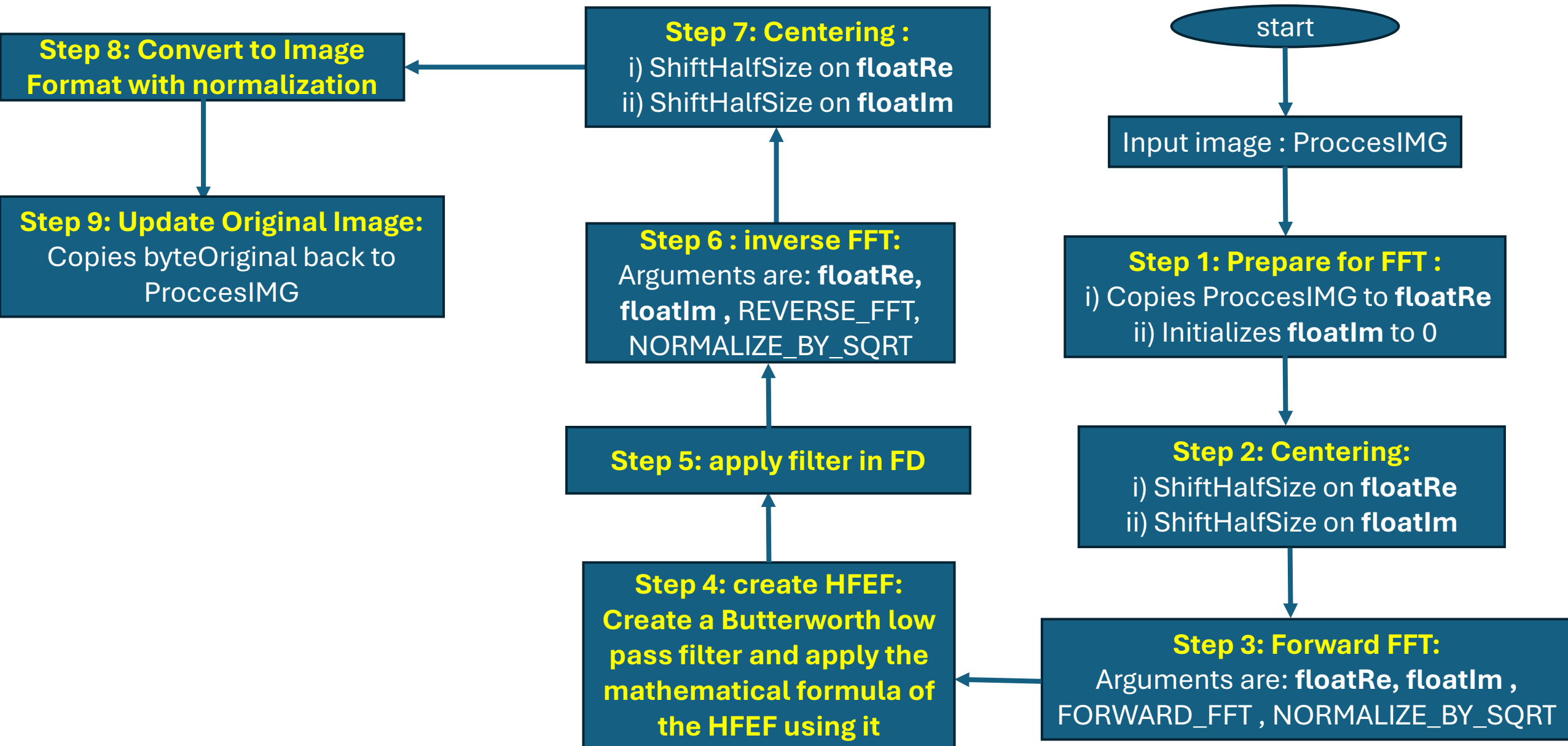
Pay attention : the structure of that function is very similar to the structure of the work() function – the only difference between them is step 4 which is the creation of the filter. We did not manage to combine the functions into one generic function due to lack of time

```

449 // Step 7: Convert and store the final result
450 OptimalConvert(floatRe, byteOriginal);
451
452 // Step 8: Copy the result back to ProccesIMG
453 for (int i = 0; i < NUMBER_OF_ROWS; i++)
454 {
455     for (int j = 0; j < NUMBER_OF_COLUMNS; j++)
456     {
457         ProccesIMG[i][j] = byteOriginal[i][j];
458     }
459 }
460

```

Block diagram of the WorkHFEF () function:



Some additional notes about the processes that related both to WorkHFEF() and work()

```
// Step 7: Convert and store the final result  
OptimalConvert(floatRe, byteOriginal);
```

Converts the real part of the spatial domain image (floatRe) to an unsigned char format.

Purpose: Maps floating-point pixel intensities to the 0-255 range suitable for image representation.

```
// Step 1: Prepare the image for FFT  
for (int i = 0; i < NUMBER_OF_ROWS; i++)  
{  
    for (int j = 0; j < NUMBER_OF_COLUMNS; j++)  
    {  
        floatRe[i][j] = ProccesIMG[i][j];  
        floatIm[i][j] = 0; // Ensure imaginary part is initialized to 0  
    }  
}
```

Purpose: Prepares the image data for the Fourier Transform by separating real and imaginary components.

```
// Step 2: Apply centering to the image  
ShiftHalfSize(floatRe);  
ShiftHalfSize(floatIm);
```

Repositions the zero-frequency (DC component) from the top-left corner to the center of the frequency domain.

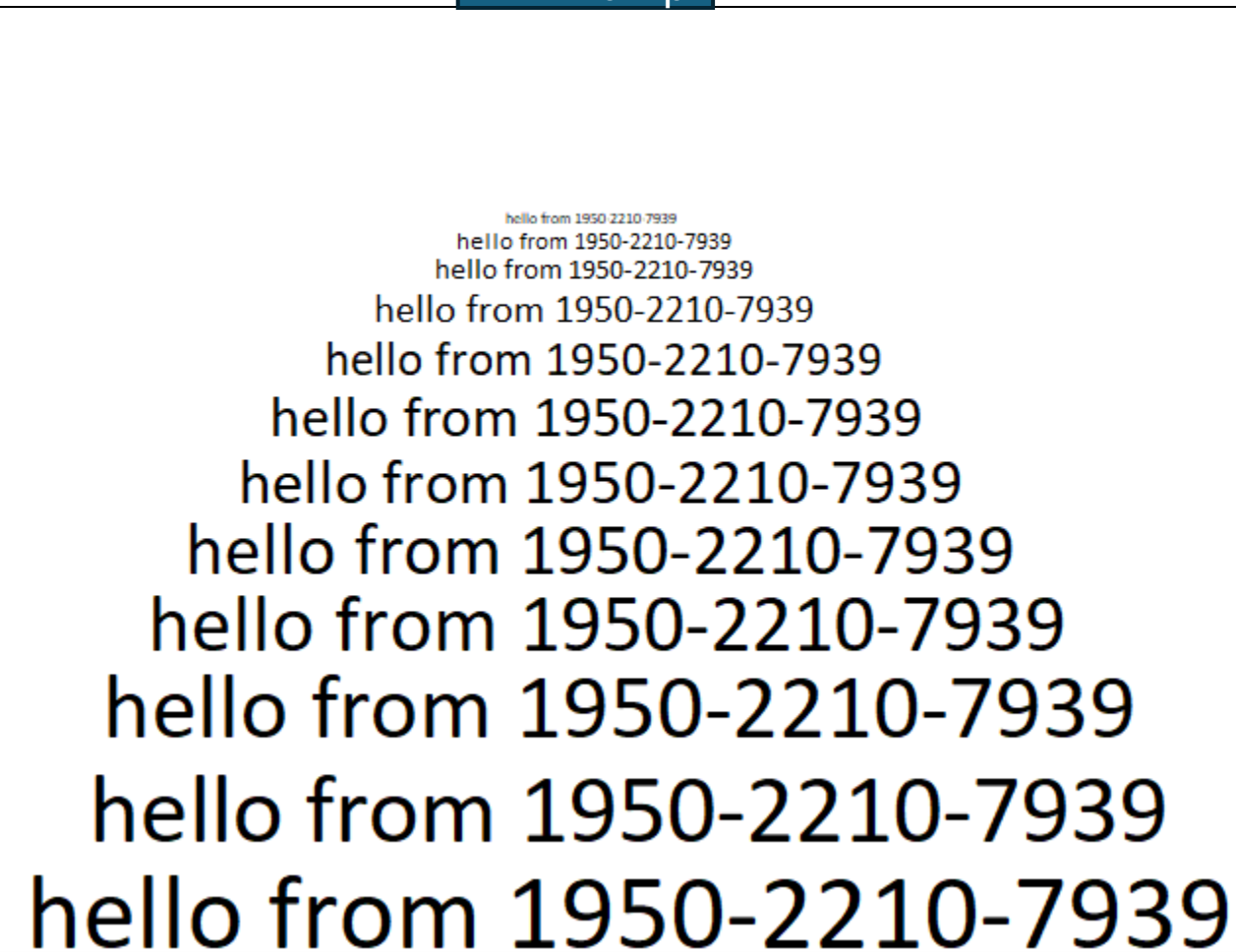
Purpose: Centers the frequency components, making it easier to apply frequency-based filters.

MP2.1 – the input images we are going to test – part 1

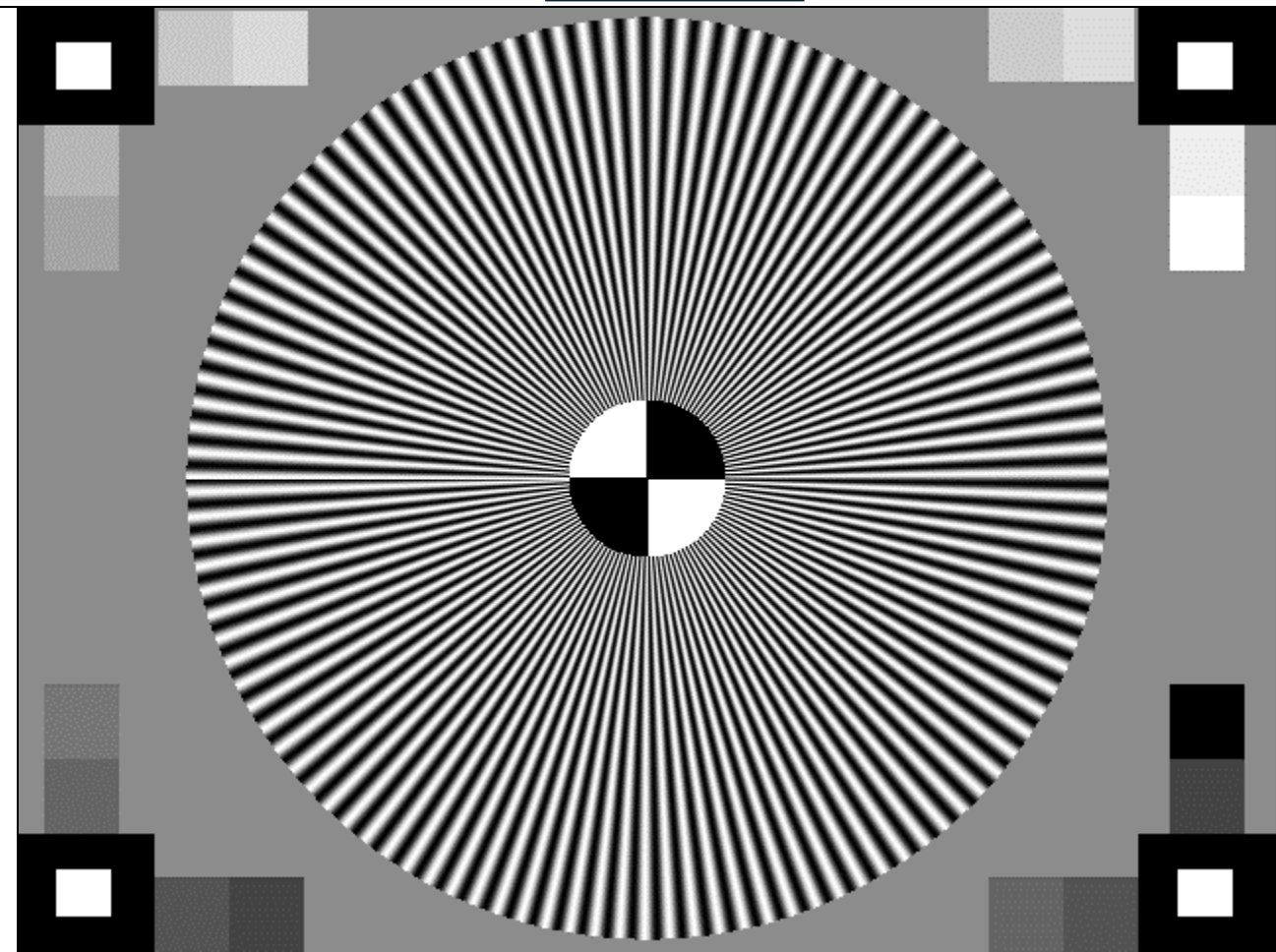
23

1. (1 point) By using MS Paint
create Gray BMP VGA Image (640x480) Tim1.bmp containing 12 lines of the text:
“Hello from **ABCD-EFGH-IJKL**” (Replace RED)
* Font size of every line is differ (from small (say, 8 or less) to large)
7 (6 points) Recreate grayImage13bmp from Assignment 13 (TV Test Image) with sizes 640x480.

Tim1.bmp



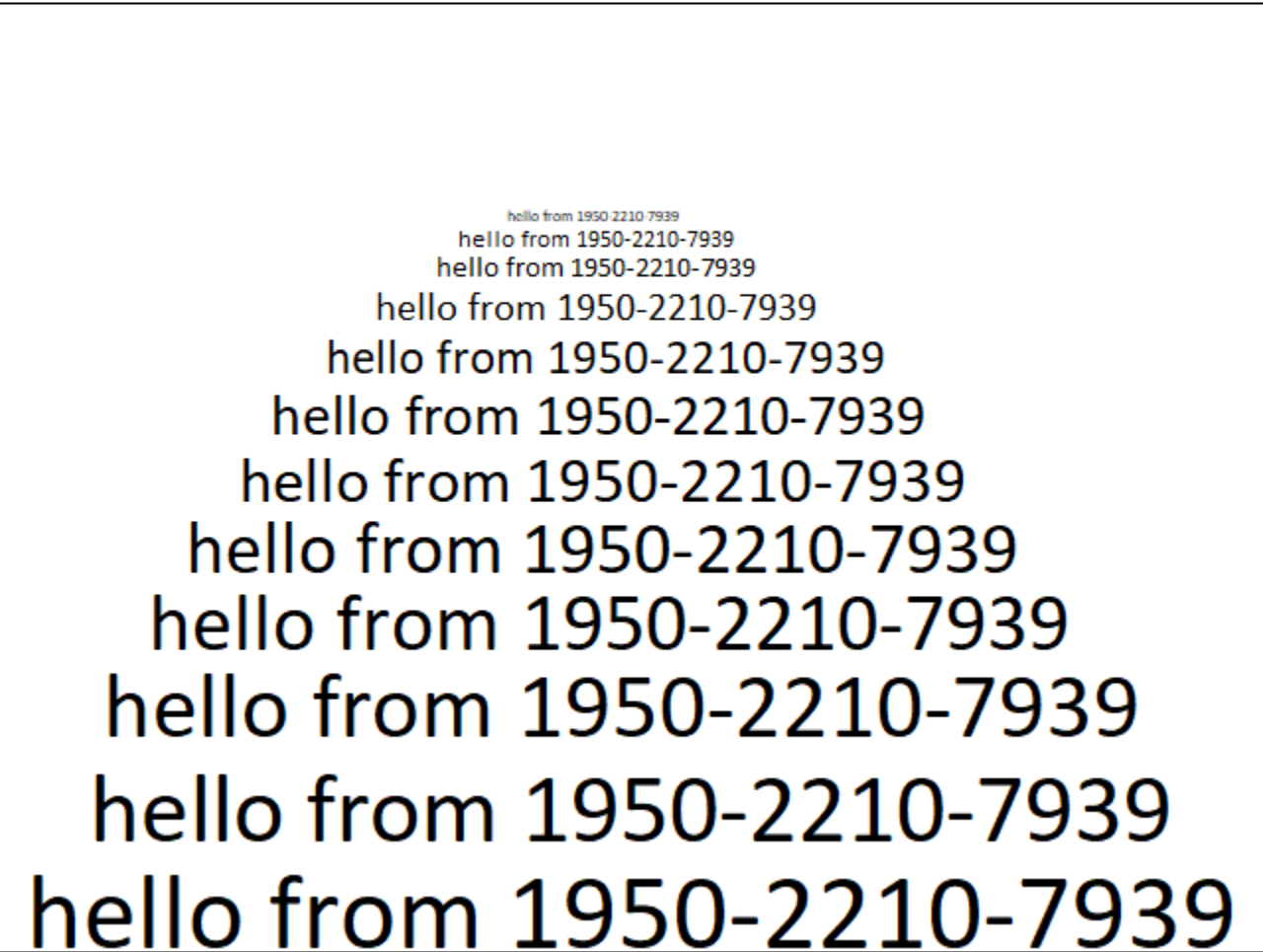
Tim2.bmp



MP2.1 – the input images we are going to test – **part 2**

24

Tim1.bmp



hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939
hello from 1950-2210-7939

Image

Dimensions	640 x 480
Width	640 pixels
Height	480 pixels
Bit depth	24

File

Name	Tim1.bmp
Item type	BMP File



The Tim1.bmp is comprised of repeated text 12 times. The font size of the text is declining from 33 to 5 in jumps of 2/3.

Regarding Tim2.bmp – it is a modification of the result of assignment 13 to be in size 640x480

Image

Dimensions	640 x 480
Width	640 pixels
Height	480 pixels
Bit depth	8

File

Name	Tim2.bmp
Item type	BMP File

MP2.2 – blurring of the images using two sizes of fast gaussians 1D filters in convolution (TD) - part 1

This is the relevant part of the main function. The functions that being used in that part is the **DoGaussianFiltration** which is a convolution filtering that uses the separability feature of the gaussian

In the first part the filter size is 5 and in the second part the filter size is 7.

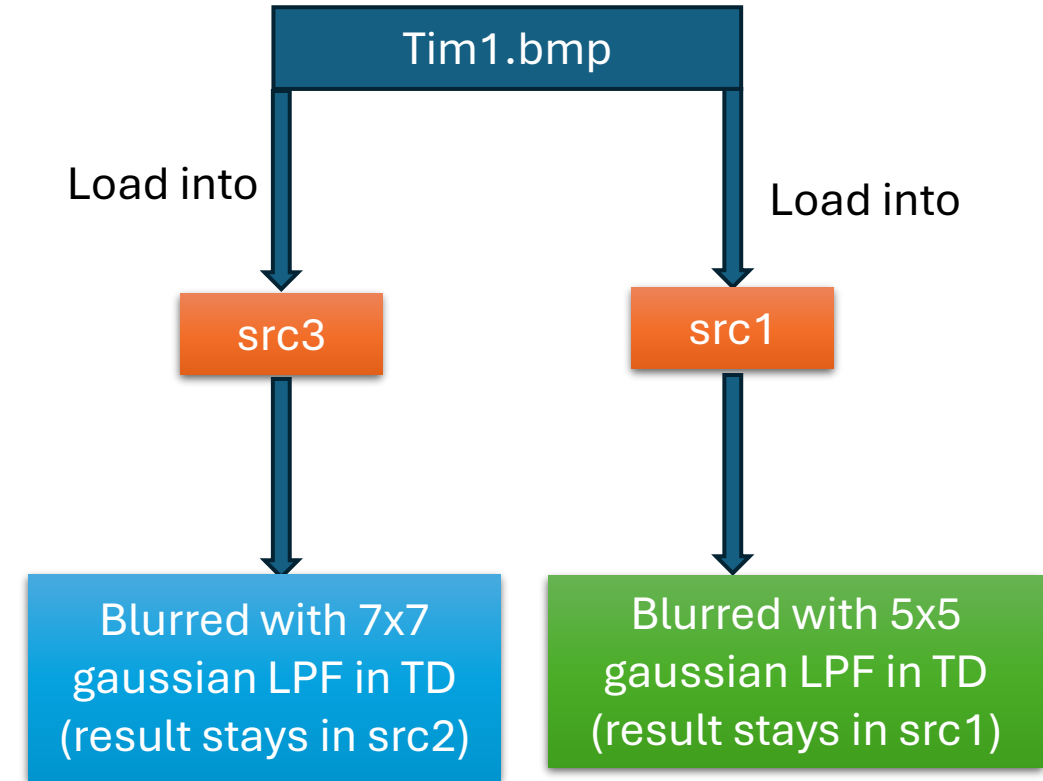
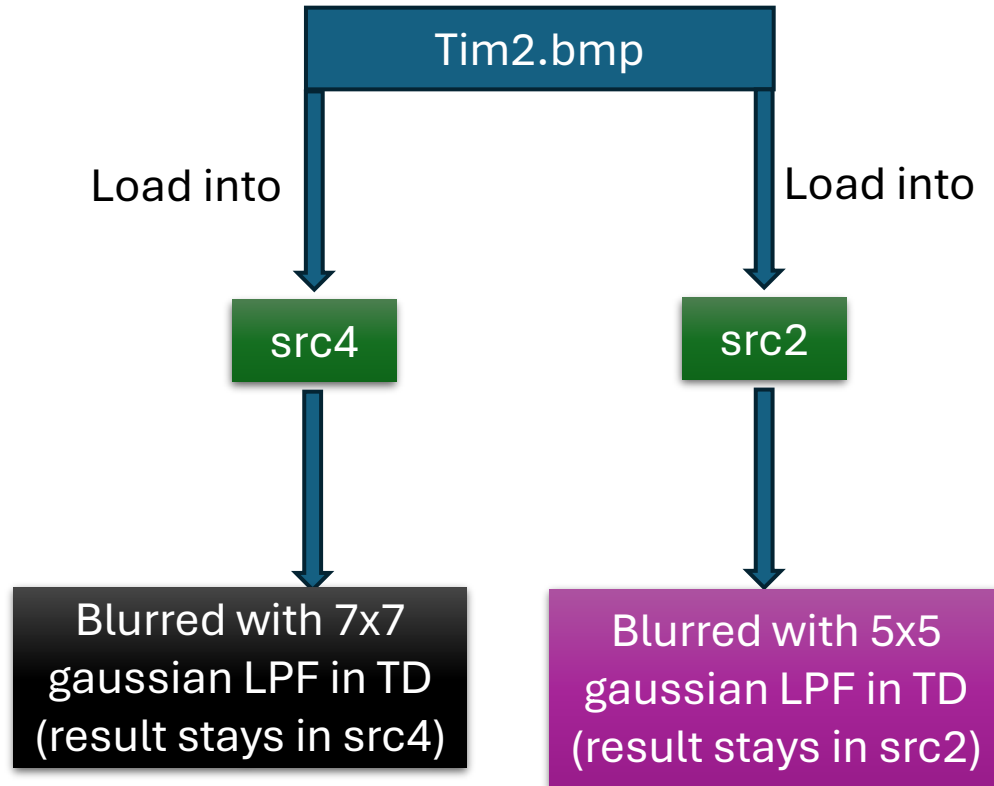
The offsets that we provided to the given kernels to fulfil the requirements – we reached them in trial and error until we got the optimal offsets that stands for the demands

```

489 void main()
490 {
491     //part 2 of the task - "By using reworked 2D 5x5 and 7x7 filters BLUR Image Tim1.bmp by using convolution with Fast Gaussian 1D Filters"
492     cout << "Gaussian filtration for blurring the input Tim1.bmp and Tim2.bmp images" << endl;
493
494     // Apply a 5x5 Gaussian filter
495     const int FILTER_HALF_SIZE_5 = 2;
496     double offset = 0.23;
497     const int FILTER_SIZE_5 = 2 * FILTER_HALF_SIZE_5 + 1;
498     double filter_5[FILTER_SIZE_5] = { 7 / 273 + offset, 26 / 273 + offset, 41 / 273 + offset, 26 / 273 + offset, 7 / 273 + offset };
499
500     LoadGrayImageFromTrueColorBmpFile(src1, "Tim1.bmp"); // Load the image
501     DoGaussianFiltration(&src1[0][0], filter_5, FILTER_SIZE_5); // Perform the filtration
502     StoreGrayImageAsGrayBmpFile(src1, "Tim1LPF5c.bmp"); // Save the blurred image with 5x5 filter
503     ...
504     // Apply a 7x7 Gaussian filter
505     const int FILTER_HALF_SIZE_7 = 3;
506     const int FILTER_SIZE_7 = 2 * FILTER_HALF_SIZE_7 + 1;
507     double offset2 = 0.16;
508     double filter_7[FILTER_SIZE_7] = { 2 / 1003 + offset2, 22 / 1003 + offset2, 97 / 1003 + offset2, 159 / 1003 + offset2, 97 / 1003 + offset2,
509                                         22 / 1003 + offset2, 2 / 1003 + offset2 };
510
511     LoadGrayImageFromTrueColorBmpFile(src3, "Tim1.bmp"); // Reload the original image
512     DoGaussianFiltration(&src3[0][0], filter_7, FILTER_SIZE_7); // Perform the filtration
513     StoreGrayImageAsGrayBmpFile(src3, "Tim1LPF7c.bmp"); // Save the blurred image with 7x7 filter

```

MP2.2 – blurring of the images using two sizes of fast gaussians 1D filters in convolution (TD) - **part 2**



```
467 unsigned char src1[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];  
468 unsigned char src2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];  
469 unsigned char src3[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];  
470 unsigned char src4[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];  
// src1-result of the blurring convolution between 5x5 filter & Tim1  
// src2-result of the blurring convolution between 5x5 filter & Tim2  
// src3-result of the blurring convolution between 7x7 filter & Tim1  
// src4-result of the blurring convolution between 7x7 filter & Tim2
```

MP2.2 – blurring of the images using two sizes of fast gaussians 1D filters in convolution (TD) - part 1

Tim1LPF5c.bmp

filter gives different results in the different kernels sizes (different offset)

Tim1LPF7c.bmp



As one can see-both of the images stands for the demand that : after the blurring at least 2 lines will be hardly readable and 3 lines unreadable

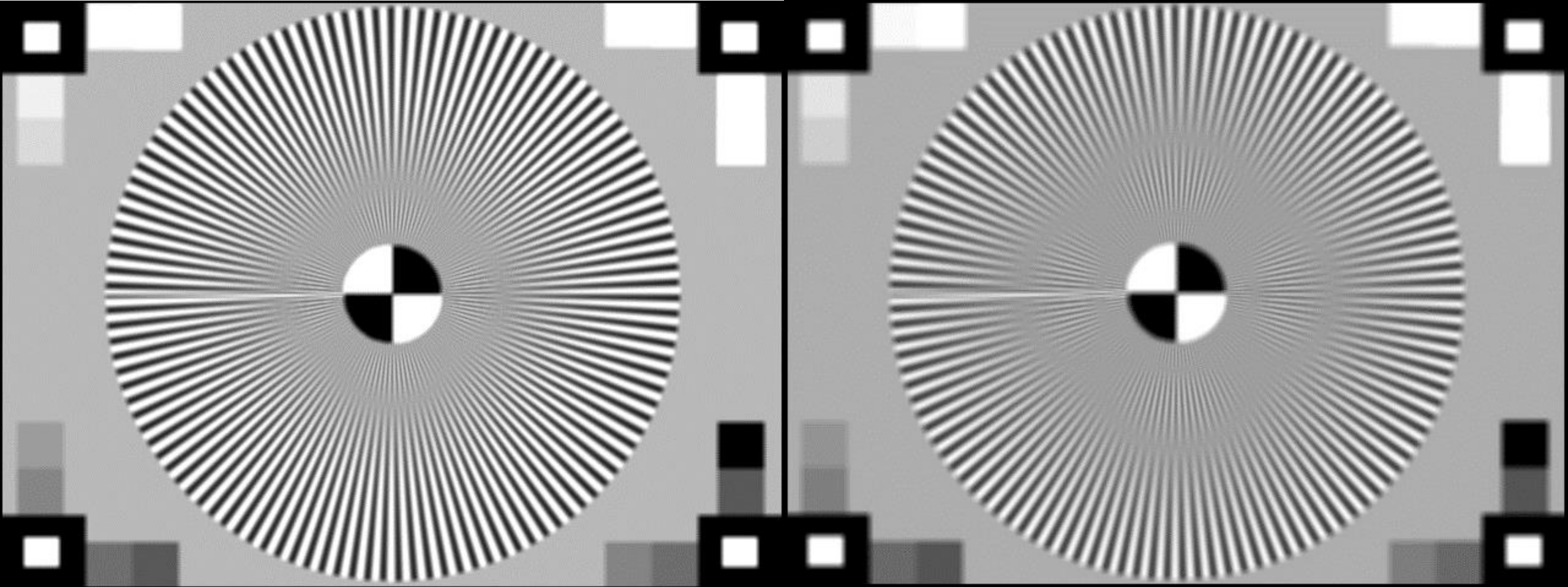
The presented images are at their original size (640x480)

MP2.2 – blurring of the images using two sizes of fast gaussians 1D filters in convolution (TD) - part 2

Tim2LPF5c.bmp

The filter gives different results in the different kernels sizes (different offset)

Tim2LPF7c.bmp



We can see that in the result of the 7x7 filter (Tim2LPF7c.bmp) there is much more data loss and in both images there is no aliasing in the image

MP2.3 – blurring of the images using one size of gaussians filters using FFT (FD) - **part 1**

29

In MP2.3 we had to filter in FD – hence using the FFT process using the work() function to blur the input image using gaussian filter.

As a gaussian filter we use the known function of CreateGreyGaussian and needed to adjust its parameters for blurring at least 6 lines that will be hardly readable and unreadable

```
464 unsigned char ProccesIMG1[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; //ProccesIMG1 are the results of the blurring FFT of Tim1.bmp and then enhancing
465 unsigned char ProccesIMG2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; //ProccesIMG2 are the results of the blurring FFT of Tim2.bmp and then enhancing
```

```
577 //***** part 3 of the task - "Blur test Images by using FFT using Gaussian Filter in FD. " *****
578
579 // Load and process the Tim1.bmp image
580 LoadGrayImageFromTrueColorBmpFile(ProccesIMG1, "Tim1.bmp");
581
582 /// FFT filtration to blur using gaussian filter
583 Work(ProccesIMG1, 375);
584 StoreGrayImageAsGrayBmpFile(ProccesIMG1, "Tim1LPFF.bmp");
```

```
596 ////***** repeat the steps before on Tim2.bmp given image *****
597
598 // Load and process the Tim2.bmp image
599 LoadGrayImageFromGrayBmpFile(ProccesIMG2, "Tim2.bmp");
600
601 /// FFT filtration using gaussian filter
602 Work(ProccesIMG2, 500);
603 StoreGrayImageAsGrayBmpFile(ProccesIMG2, "Tim2LPFF.bmp");
```

The size of the filter that we provided to the work() to fulfil the requirements – we reached them in trial and error until we got the optimal sizes that stands for the demands

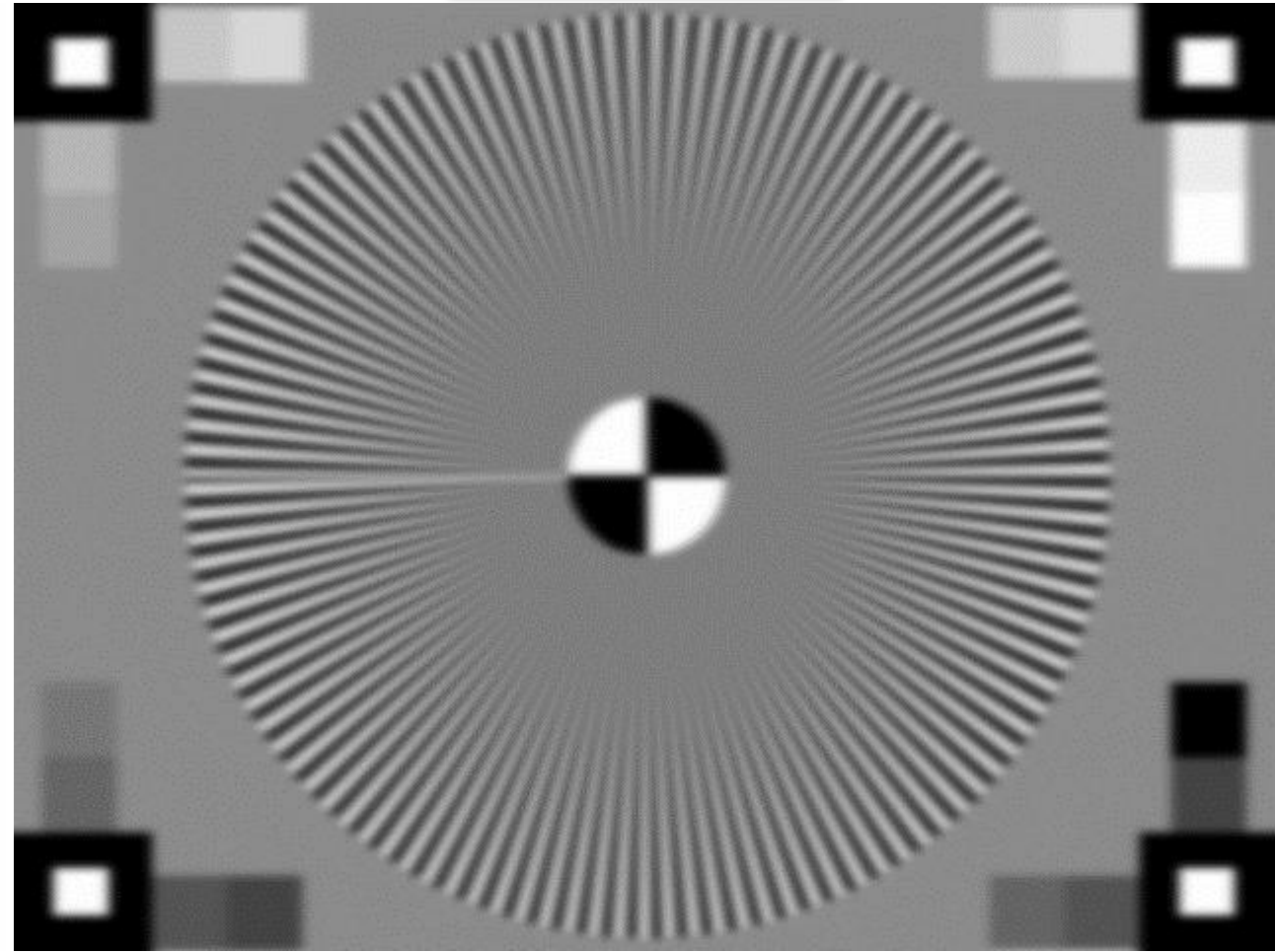
MP2.3 – blurring of the images using one size of gaussians filters using FFT (FD) - **part 2**

30

Tim1LPFF.bmp



Tim2LPFF.bmp



The presented images are at
their original size (640x480)

MP2.4 – restoration of the blurred images created using convolution using at least two HFEF - **part 0**

31

We created 3 restoration types for each blurred image (src#1,2,3,4).

We checked each image with 3 different high frequency enhancement filters – Laplacian , high boost and unsharp masking when for each type we checked for 3 different sizes of kernel – 3 , 5 & 7.

The total number of tested scenarios is $3*3*3*3=27$ -> out of them we picked and left in the code (and will present) only the 9 relevant and best from our perspective .

Since the results of the Laplacian were not satisfying enough – we excluded them from that presentation.

The names of the destination's variables are named after 3 parameters:

- 1) src number
- 2) filter size (3/5/7)
- 3) option number

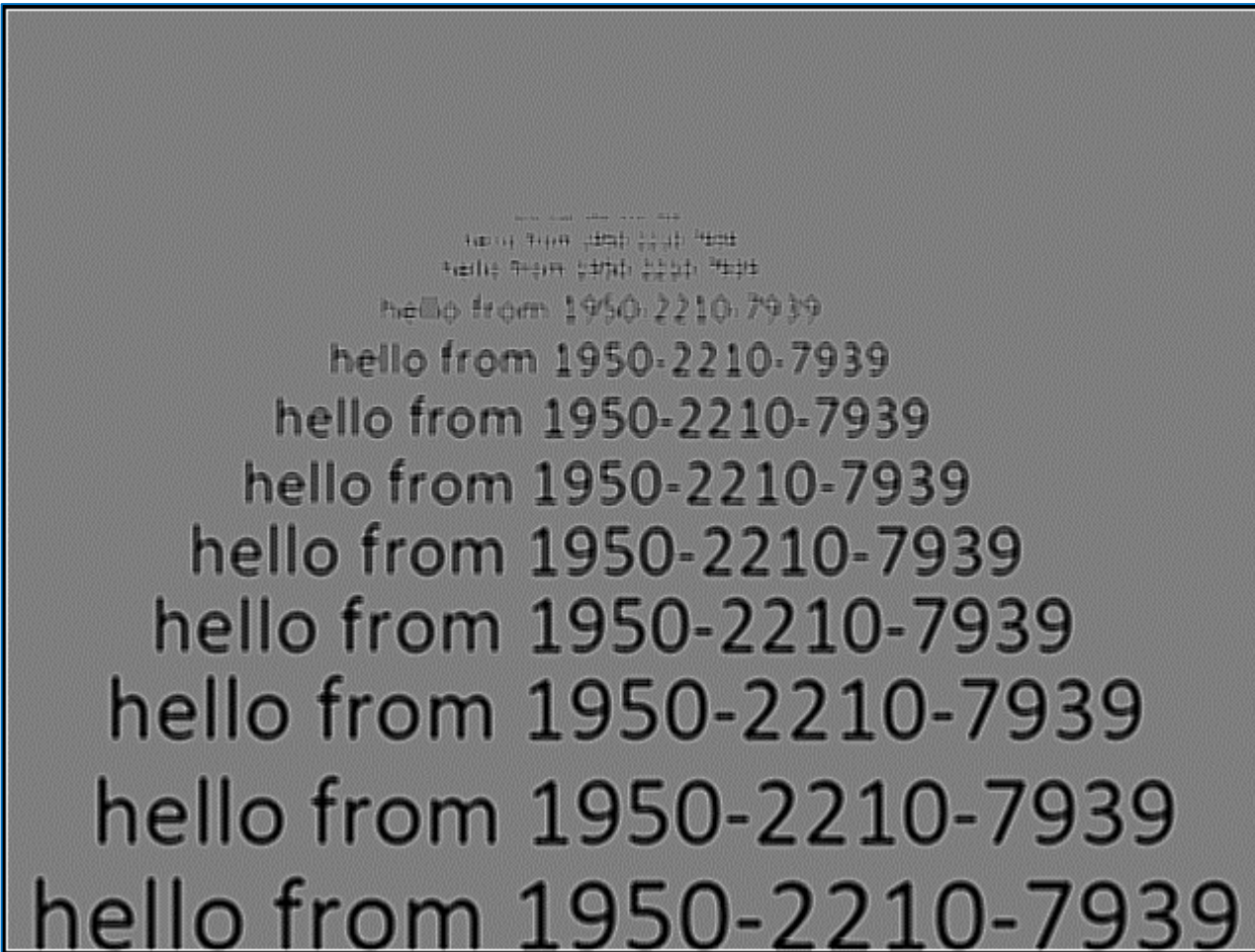
```
472 unsigned char dst13_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest13-result of the restore convolution between 3x3 kernel & src1
473 unsigned char dst15_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest15-result of the restore convolution between 5x5 kernel & src1
474 unsigned char dst17_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest17-result of the restore convolution between 7x7 kernel & src1
475
476 unsigned char dst33_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest33-result of the restore convolution between 3x3 kernel & src3
477 unsigned char dst35_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest35-result of the restore convolution between 5x5 kernel & src3
478 unsigned char dst37_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest37-result of the restore convolution between 7x7 kernel & src3
479
480 unsigned char dst23_option3[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest23-result of the restore convolution between 3x3 kernel & src2
481 unsigned char dst25_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest15-result of the restore convolution between 5x5 kernel & src2
482 unsigned char dst27_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest27-result of the restore convolution between 5x5 kernel & src2
483
484 unsigned char dst43_option3[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest43-result of the restore convolution between 3x3 kernel & src4
485 unsigned char dst45_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest45-result of the restore convolution between 5x5 kernel & src4
486 unsigned char dst47_option2[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // dest47-result of the restore convolution between 7x7 kernel & src4
```

MP2.4 – restoration of the blurred images created using convolution using at least two HPF - part 1

32

Tim1 blurred by gaussian in size 5 restored by kernel_1

Tim1LPF5CHFEF1C.bmp



Tim1 blurred by gaussian in size 7 restored by kernel_2

Tim1LPF5CHFEF2C.bmp



Tim1LPF5CHFEF3C.bmp

Tim1LPF7CHFEF1C.bmp

[illegible]

MP2.4 – restoration of the blurred images created using **convolution** using at least two HPF - **part 3**

Tim1 blurred by gaussian in size 7 restored by kernel_2

Tim1LPF7CHFEF2C.bmp



Tim1 blurred by gaussian in size 7 restored by kernel_3

Tim1LPF7CHFEF3C.bmp



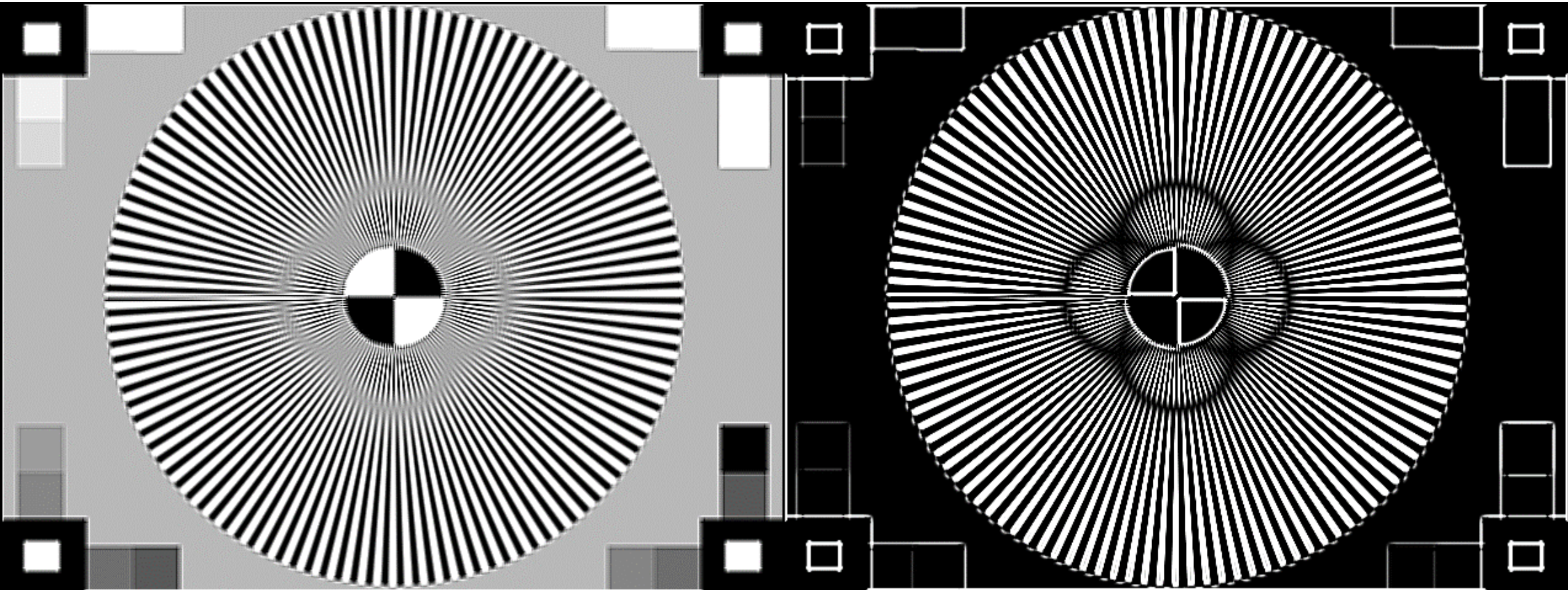
MP2.4 – restoration of the blurred images created using convolution using at least two HPF - part 4

Tim2 blurred by gaussian in size 5 restored by kernel_1

Tim2LPF5CHFEF1C.bmp

Tim2 blurred by gaussian in size 5 restored by kernel_2

Tim2LPF5CHFEF2C.bmp



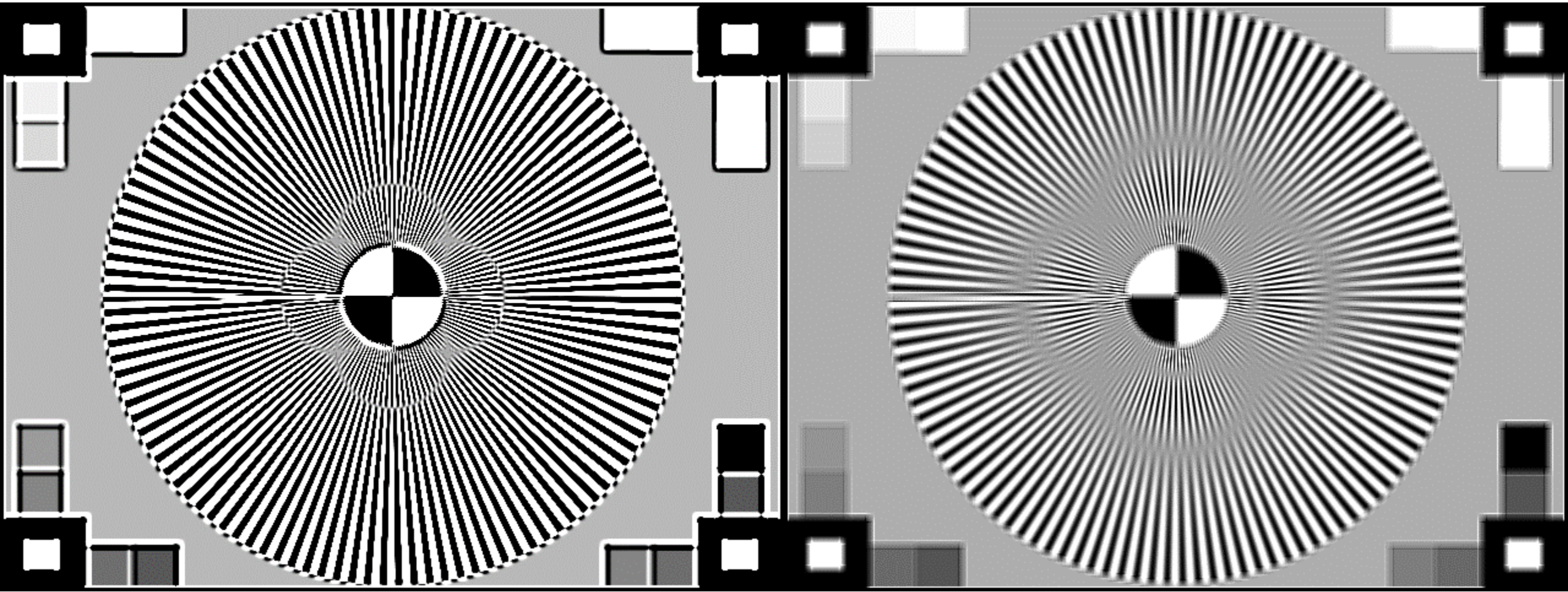
MP2.4 – restoration of the blurred images created using convolution using at least two HPF - part 5

Tim2 blurred by gaussian in size 5 restored by kernel_3

Tim2LPF5CHFEF3C.bmp

Tim2 blurred by gaussian in size 7 restored by kernel_1

Tim2LPF7CHFEF1C.bmp



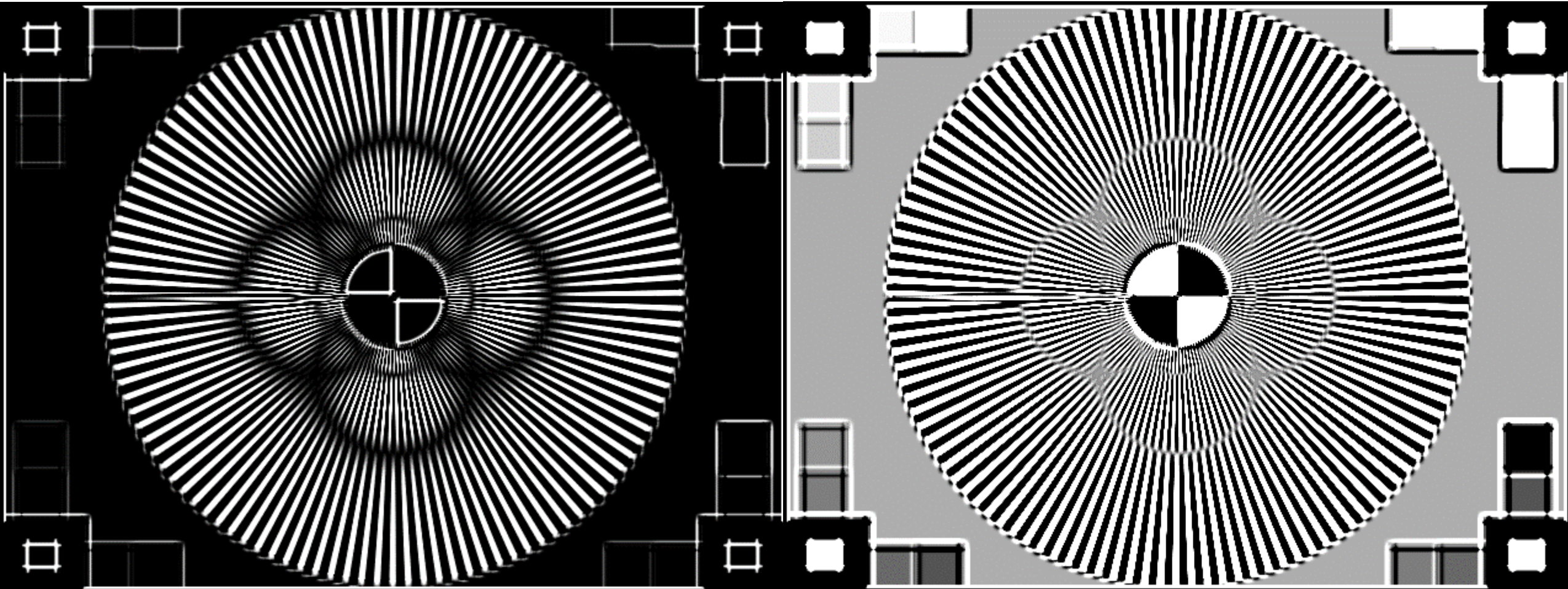
MP2.4 – restoration of the blurred images created using convolution using at least two HPF - part 5

Tim2 blurred by gaussian in size 7 restored by kernel_2

Tim2LPF7CHFEF2C.bmp

Tim2 blurred by gaussian in size 7 restored by kernel_3

Tim2LPF7CHFEF3C.bmp



MP2.4 – restoration of the blurred images created using convolution using at least two HPF - part 6

38

The calls to the filtration using convolution from the main

```
515 //***** part 4 of the task - try to enhance/restore the blurred images using convolution in TD *****
516
517 DoFiltationByConvolution(src1, dst13_option2, &kernel_3x3_option2_High_Boost_Filter[0][0], 3);
518 StoreGrayImageAsGrayBmpFile(dst13_option2, "Tim1LPF5CHFEF1C.bmp");
519
520 //-----//
521 DoFiltationByConvolution(src1, dst15_option2, &kernel_5x5_option2_High_Boost_Filter[0][0], 5);
522 StoreGrayImageAsGrayBmpFile(dst15_option2, "Tim1LPF5CHFEF2C.bmp");
523
524 //-----//
525 DoFiltationByConvolution(src1, dst17_option2, &kernel_7x7_option2_High_Boost_Filter[0][0], 7);
526 StoreGrayImageAsGrayBmpFile(dst17_option2, "Tim1LPF5CHFEF3C.bmp");
527
528
529 DoFiltationByConvolution(src3, dst33_option2, &kernel_3x3_option2_High_Boost_Filter[0][0], 3);
530 StoreGrayImageAsGrayBmpFile(dst33_option2, "Tim1LPF7CHFEF1C.bmp");
531
532 //-----//
533 DoFiltationByConvolution(src3, dst35_option2, &kernel_5x5_option2_High_Boost_Filter[0][0], 5);
534 StoreGrayImageAsGrayBmpFile(dst35_option2, "Tim1LPF7CHFEF2C.bmp");
535
536 //-----//
537 DoFiltationByConvolution(src3, dst37_option2, &kernel_7x7_option2_High_Boost_Filter[0][0], 7);
538 StoreGrayImageAsGrayBmpFile(dst37_option2, "Tim1LPF7CHFEF3C.bmp");
```

MP2.4 – restoration of the blurred images created using convolution using at least two HPF – part 7

The calls to the filtration using convolution from the main - continue

```
549 //-----//
550 DoFiltationByConvolution(src2, dst23_option3, &kernel_3x3_option3_Unsharp_Masking_Filter[0][0], 3);
551 StoreGrayImageAsGrayBmpFile(dst23_option3, "Tim2LPF5CHFEF1C.bmp");
552
553 //-----//
554 DoFiltationByConvolution(src2, dst25_option2, &kernel_5x5_option2_High_Boost_Filter[0][0], 5);
555 StoreGrayImageAsGrayBmpFile(dst25_option2, "Tim2LPF5CHFEF2C.bmp");
556
557 //-----//
558 DoFiltationByConvolution(src2, dst27_option2, &kernel_7x7_option2_High_Boost_Filter[0][0], 7);
559 StoreGrayImageAsGrayBmpFile(dst27_option2, "Tim2LPF5CHFEF3C.bmp");
560
561 //-----//
562
563 DoFiltationByConvolution(src4, dst43_option3, &kernel_3x3_option3_Unsharp_Masking_Filter[0][0], 3);
564 StoreGrayImageAsGrayBmpFile(dst43_option3, "Tim2LPF7CHFEF1C.bmp");
565
566 //-----//
567 DoFiltationByConvolution(src4, dst45_option2, &kernel_5x5_option2_High_Boost_Filter[0][0], 5);
568 StoreGrayImageAsGrayBmpFile(dst45_option2, "Tim2LPF7CHFEF2C.bmp");
569
570 //-----//
571 DoFiltationByConvolution(src4, dst47_option2, &kernel_7x7_option2_High_Boost_Filter[0][0], 7);
572 StoreGrayImageAsGrayBmpFile(dst47_option2, "Tim2LPF7CHFEF3C.bmp");
573
```

MP2.4 – restoration of the blurred images created using convolution using at least two HPF - part 8

```
47 double kernel_3x3_option2_High_Boost_Filter[FILTER_HALF_HEIGHT + 1][FILTER_HALF_HEIGHT + 1] =  
48 {  
49     {-0.25, -0.25, -0.25},  
50     {-0.25, 2.5, -0.25},  
51     {-0.25, -0.25, -0.25}  
52 };
```

```
54 double kernel_3x3_option3_Unsharp_Masking_Filter[FILTER_HALF_HEIGHT + 1][FILTER_HALF_HEIGHT + 1] =  
55 {  
56     {-1, -1, -1},  
57     {-1, 9, -1},  
58     {-1, -1, -1}  
59 };
```

These kernels were generated by
ChatGPT preview o1

MP2.4 – restoration of the blurred images created using convolution using at least two HPF - part 8

```
71 double kernel_5x5_option2_High_Boost_Filter[2 * FILTER_HALF_HEIGHT + 1][2 * FILTER_HALF_HEIGHT + 1] =
72 {
73     {-1, -1, -1, -1, -1},
74     {-1, -1, -1, -1, -1},
75     {-1, -1, 24, -1, -1},
76     {-1, -1, -1, -1, -1},
77     {-1, -1, -1, -1, -1}
78 };
79
```

These kernels were generated by
ChatGPT preview o1

```
100 double kernel_7x7_option2_High_Boost_Filter[2 * FILTER_HALF_HEIGHT + 3][2 * FILTER_HALF_HEIGHT + 3] =
101 {
102     {-1, -1, -1, -1, -1, -1, -1},
103     {-1, -1, -1, -1, -1, -1, -1},
104     {-1, -1, -1, -1, -1, -1, -1},
105     {-1, -1, -1, 49, -1, -1, -1},
106     {-1, -1, -1, -1, -1, -1, -1},
107     {-1, -1, -1, -1, -1, -1, -1},
108     {-1, -1, -1, -1, -1, -1, -1}
109 };
```

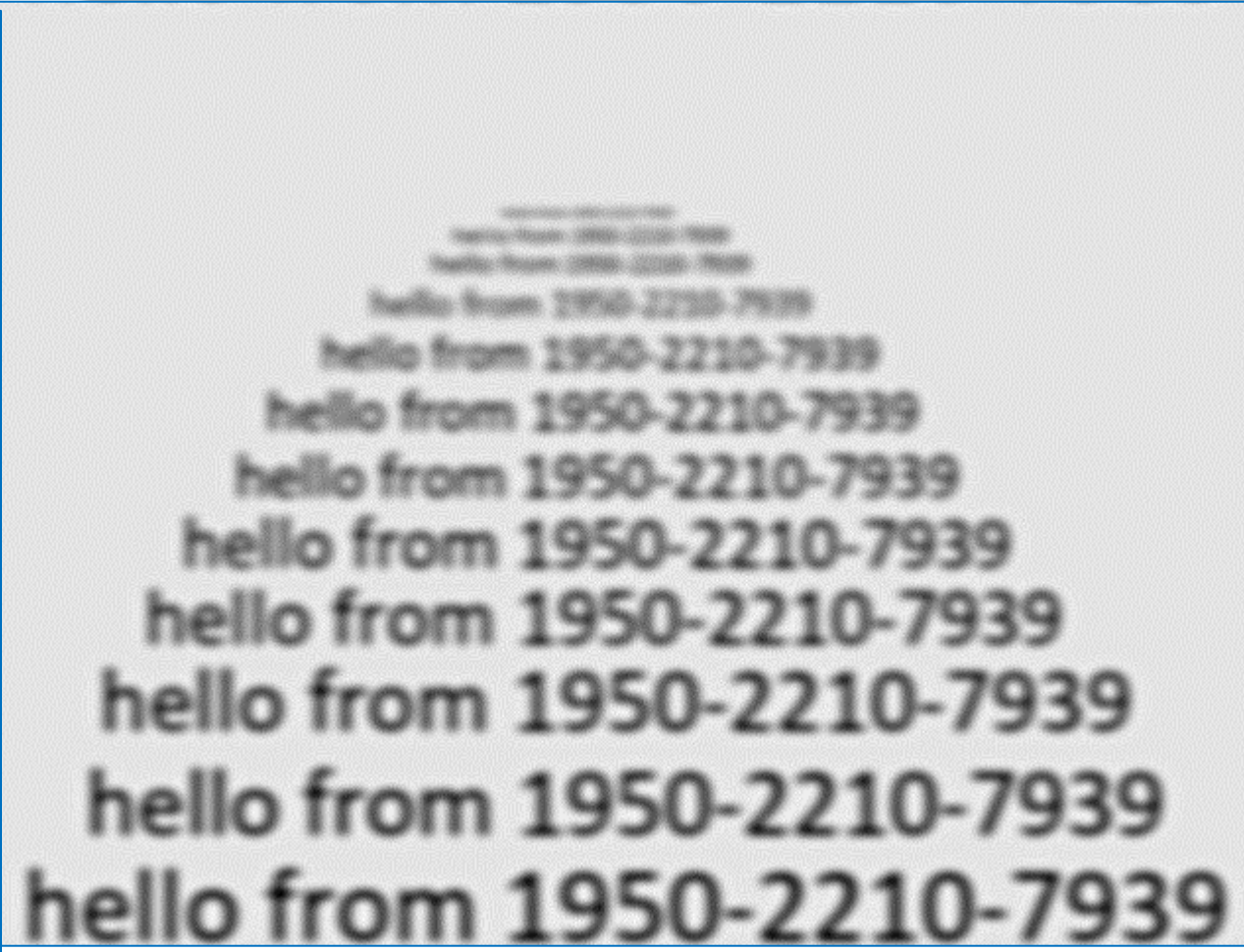
MP2.5 – restoration of the blurred images created using FFT using one “enhancing HPF “- **part 1**

42

Tim1LPFF.bmp – the blurred image through FFT



Tim1LPFFHFEff.bmp– the restored image through FFT

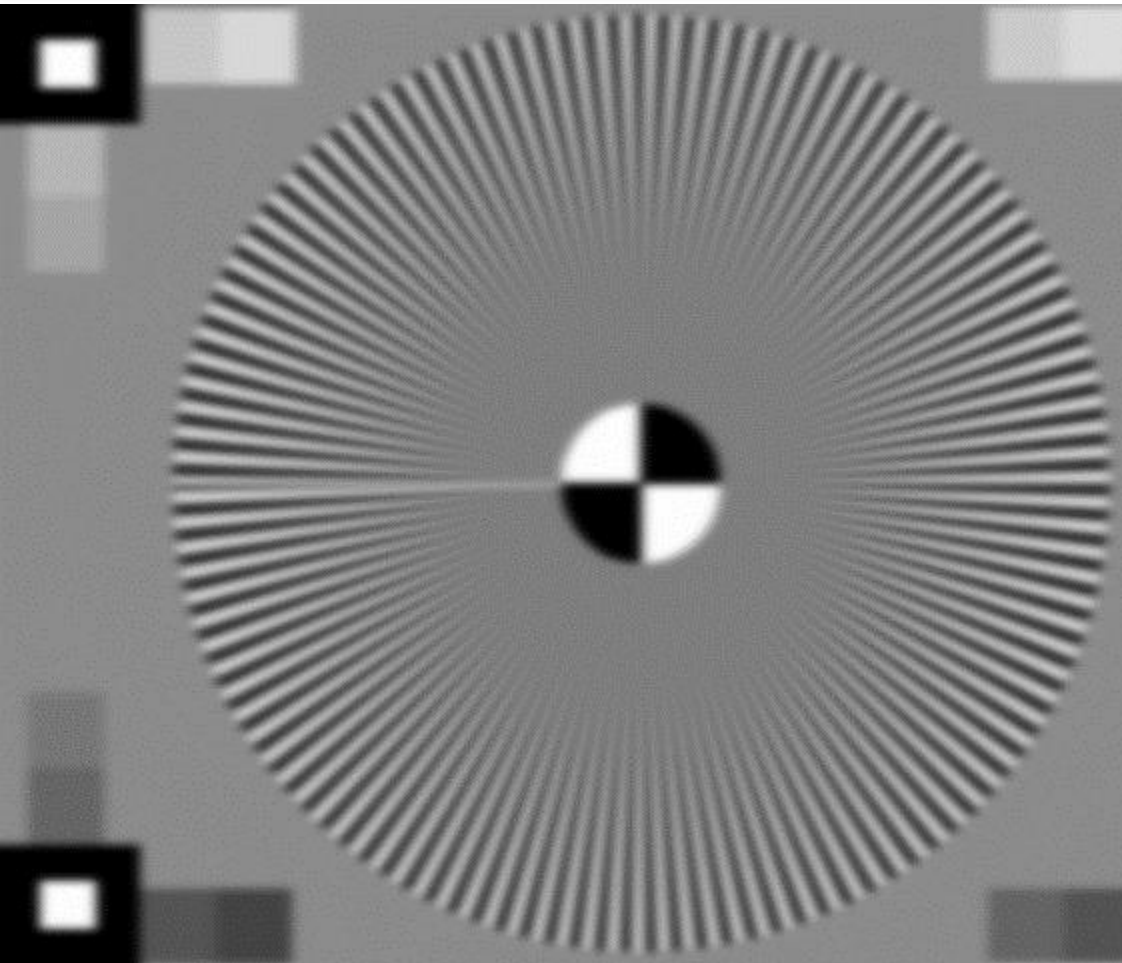


We can see that from line 5 there is a separation between letters in the “hello from” which indicates on good restoration

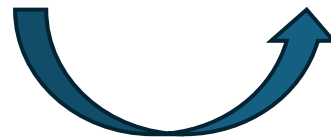
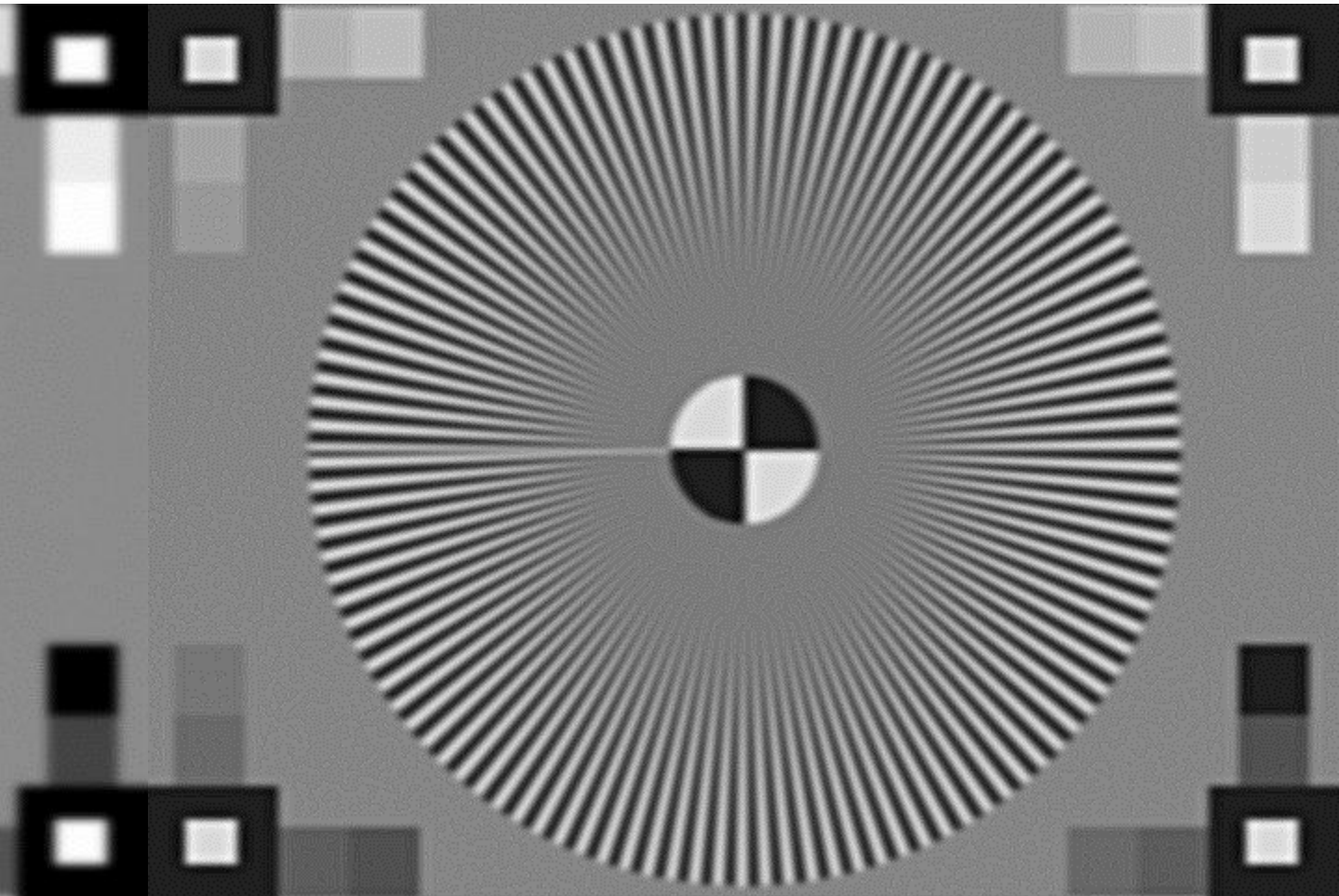
MP2.5 – restoration of the blurred images created using FFT using one “enhancing HPF “- **part 2**

43

Tim2LPFF.bmp – the blurred image through FFT



Tim2LPFFHFEff.bmp – the restored image through FFT



The enhanced image is much sharper in the edges of the circle and the initial blurring in the center has now sharper edges

MP2.5 – restoration of the blurred images created using FFT using one “enhancing HPF “- **part 3**

44

```
584 // Step 4: Create the HFEF
585 int cutoff_tim1 = 500;          // Adjust as needed
586 int order_tim1 = 1;            // Adjust as needed
587 float boost_factor_tim1 = 150.0f;
588 /// FFT filtration using HPEF to restore (enhance) the blurred image - section 5
589 WorkHFEF(ProccesIMG1, cutoff_tim1, order_tim1, boost_factor_tim1);
590
591
592 StoreGrayImageAsGrayBmpFile(ProccesIMG1, "Tim1LPFFHFEFf.bmp");
```

In both cases the parameters were created by a trial and error until reaching the optimal results (maximum enhancement)

```
603 // Step 4: Create the HFEF
604 int cutoff_tim2 = 500;          // Adjust as needed
605 int order_tim2 = 1;            // Adjust as needed
606 float boost_factor_tim2 = 150.0f;
607 /// FFT filtration using HPF to restore (enhance) the blurred image - section 5
608 WorkHFEF(ProccesIMG2, cutoff_tim2, order_tim2, boost_factor_tim2);
609 StoreGrayImageAsGrayBmpFile(ProccesIMG2, "Tim2LPFFHFEFf.bmp");
```

MP2.6 – bibliography

ChatGPT – thanks to the chat for a lot of wonderful work

Course lecturers – we used the lecturers and the functions in it